How to loop in SQL

The SQL standard enables the query author to …

CTEs: Binding intermediate results

As queries get more complex, the exemplary query author wants to keep their code well organized. Using too many subqueries often result in hard-to-read code. What if SQL had a way to define intermediate queries beforehand and bind their result tables to names, similar to variables in imperative programming languages?

SQLs way of doing this is called the **Common Table Expression** (CTE). The general outline is shown in Listing 1. A CTE is defined via the `WITH` clause, followed by the cte name, followed by an arbitrarily large list of columns. The inner query is being evaluated before the outer query, and its result is being stored into a table named `cte_name`. This table has the columns specified in the CTE definition; their data types are derived from the inner query. The outer query can then reference this table.

```
WITH cte_name(col1, col2, ...) AS (
  <inner query>
)
<outer query>
```

General outline of a CTE

```
WITH one_plus_one(x) AS (
    SELECT 1+1
)
SELECT x * 2
FROM one_plus_one;
```

Example: Calculating $(1 + 1) \cdot 2$ using a CTE

Listing 1: CTE outline and example.

`WITH RECURSIVE`: Making SQL turing-complete

In order for SQL to become turing complete, some kind of iteration mechanism had to be added to the language's standard. Together with CTEs, SQL:1999 introduced *recursive CTEs*, which expand the general CTE by allowing self-reference within the inner query.

See Listing 2 for the general layout and an example. As we are dealing with a recursive query now, we need to provide an initial case and a recursive step. The latter needs a break condition in order to avoid infinite recursion; in this example, the break condition is `WHERE n < 10`.

```
WITH RECURSIVE cte_name(col1, col2, ...)
AS (
  <initial case>

  UNION ALL

  <recursive step>
)
<outer query>
```

General outline of a recursive CTE

```
WITH RECURSIVE pow2(n, x) AS (
    SELECT 1, 2

    UNION ALL

    SELECT n+1, x*2
    FROM pow2
    WHERE n < 10
)
SELECT n, x
FROM pow2;
```

Example: Calculating 2^{10} using a recursive CTE.

Listing 2: Recursive CTE outline and example.

How do recursive CTEs work internally?

The problem with recursive CTEs

Have a look at the example in Listing 2 again. While this query confidently calculates 2^{10} for us, it also calculates **and stores** all intermediate results. In fact, the outer query does return all

intermediate results for us. In order to select the 10th power of 2 only, we need to explicitly select it in the outer query,

```
SELECT argmax(n, x), max(x)
FROM pow2;
```

[TODO: punkt?]

While the additional work is manageable in this example, it grows while queries get more complex, as we will see in later chapters.

But it is not just the query author who has to put in unnecessary work. Even if we select only those values that have been calculated in the last iteration, until this point, **all intermediate values** have been stored in the so-called *Union Table*. Again, with increasing complexity and amount of data, all this unnecessarily stored data can easily overwhelm our system.

Finally, there is

`USING KEY`: making loops great again